



DIEGO ALMEIDA DA SILVA

PROJETO CONTROLE DE ESTOQUE

Fundação de Apoio à Escola Técnica
Estado do Rio de Janeiro

DIEGO ALMEIDA DA SILVA

PROJETO CONTROLE DE ESTOQUE

Projeto controle de estoque
apresentado ao Curso Técnico em
Informática como requisito final para
a obtenção do título de Técnico em
Informática, sob a orientação do
Prof. André Washington.

Sistema de Controle de Estoque

INTRODUÇÃO:

Visão Geral do trabalho

Este trabalho apresenta o desenvolvimento de um **Sistema Web para Controle de Estoque**, o sistema foi estruturado seguindo o modelo de desenvolvimento **Fullstack**, o que significa que ele contempla desde a interface visual (HTML e CSS) com a qual o usuário interage, até o armazenamento e permanente das informações no banco de dados mysql através do Node.js.

A Estrutura do Sistema e o Papel do Servidor

Para que o sistema funcione de forma integrada, ele foi dividido em duas grandes partes que conversam entre si o tempo todo: o **Frontend** (a interface visual) e o **Back-end** (o servidor e o banco de dados).

A interface visual — composta pelas telas, botões e campos de digitação que aparecem no navegador — funciona como a "fachada" do sistema. É onde o usuário digita os dados do produto (como nome, preço e quantidade) e visualiza a tabela de estoque atualizada.

No entanto, encontrei alguns problemas para a conclusão deste trabalho. As páginas da internet, por questões de segurança e limitação técnica, não conseguem guardar informações para sempre por si mesmas, nem possuem autorização para mexer direto nos arquivos do meu computador, na qual estava instalado o MYSQL 8.0 Command line cliente, ou em sistemas de bancos de dados. É exatamente aí que entra a figura do **Servidor (Back-end)**.

O servidor Node.js funciona como um "intermediário inteligente" ou um motor invisível que roda em segundo plano. Quando o usuário clica no botão "Salvar" na página web, a interface envia esses dados imediatamente através da rede para o servidor local. O servidor recebe essas informações, confere se está tudo correto e se encarrega de abrir as portas do banco de dados do MYSQL para gravar o produto ali dentro de forma definitiva. O mesmo caminho acontece ao contrário: quando a página é aberta, ela pede socorro ao servidor, que vai até o banco de dados, busca a lista de mercadorias salva e a entrega mastigada para que a tela do navegador possa mostrar os itens salvos na tabela.

Essa arquitetura garante que, mesmo que o navegador seja fechado ou o computador seja desligado, os dados comerciais do trabalho permaneçam salvos, protegidos e prontos para serem acessados.

Estrutura dos arquivos criados

index.html (A Interface do Usuário / Frontend)

O arquivo index.html funciona como a "carcaça" ou a estrutura esquelética da aplicação. Ele é construído utilizando a linguagem de marcação HTML5 e é responsável por estruturar tudo o que o usuário enxerga na tela.

O que foi feito nele:

- **Formulário de Entrada:** Criação dos campos de texto (input) estruturados para receber os dados essenciais do produto: *Nome*, *Categoria*, *Preço Unitário*, *Quantidade em Estoque* e *Fornecedor*.
- **Painel de Desafio Javascript:** Uma seção de destaque projetada para exibir o indicador de **Valor Total do Estoque em R\$**, atualizado de forma dinâmica.
- **Tabela de Dados:** Estruturação da tabela com colunas bem definidas onde os produtos cadastrados são injetados na tela através do script.
- **Campo de Busca:** Inclusão de um campo de pesquisa para filtragem dinâmica de mercadorias.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<script>// Bom dia professor, aqui está configurado o idioma
padrão da página
//e o charset UTF-8 para garantir que nenhum texto ou acento
fique quebrado na tela.</script>
<meta charset="UTF-8">

<script>// Aqui esta tag viewport para que o nosso layout seja
consiga responder bem,
// se adaptando bem e funcionando tanto para celular quanto no
monitor.</script>
<meta name="viewport" content="width=device-width, initial-
scale=1.0">

<title>Controle de Produtos e Estoque</title>

<script>// Aqui foi feito a importação do arquivo 'style.css'
para separar totalmente
// a estrutura do HTML da parte da estilização visual.</script>
<link rel="stylesheet" href="style.css">
</head>

<body>
<header>
<script>// Este é o cabeçalho principal, onde eu coloco o título
em
//destaque no topo do nosso sistema de estoque.</script>
<h1>Sistema de Controle de Estoque</h1>
</header>

<main>
```

```

<script>// Esta seção principal para agrupar o formulário onde
foi capturado todos
// os dados que vão gerar o nosso objeto de produto.</script>
<section id="cadastro-container">
<h2>Cadastrar Novo Produto</h2>
<form id="form-produto">

<script>// Usei este input oculto 'hidden' para guardar o ID do
produto que vem do MySQL.
    //Ele fica invisível para o usuário, mas o JavaScript usa
para controle de chaves primárias.</script>
<input type="hidden" id="produto-id">

<script>// Professor, todos os campos de digitação abaixo
possuem o atributo 'required'
    //para forçar a validação e não permitir envios em
branco.</script>
<div>
<label for="nome">Nome do Produto:</label>
<input type="text" id="nome" required placeholder="Ex: Teclado
Mecânico">
</div>

<div>
<label for="categoria">Categoria:</label>
<input type="text" id="categoria" required placeholder="Ex:
Informática">
</div>

<div>
<script>// No preço,aqui pedicpara travar com 'step="0.01"' e
'min="0"' para aceitar apenas
//números decimais positivos, simulando perfeitamente uma entrada
de dinheiro.</script>
<label for="preco">Preço Unitário (R$):</label>
<input type="number" id="preco" step="0.01" min="0" required
placeholder="0.00">
</div>

<div>
<label for="quantidade">Quantidade em Estoque:</label>
<input type="number" id="quantidade" min="0" required
placeholder="0">
</div>

<div>
<label for="fornecedor">Fornecedor:</label>
<input type="text" id="fornecedor" required placeholder="Ex:
Distribuidora Tech">
</div>

<script>// Quando o usuário clica neste botão, ele dispara o
envio do formulário,

```

```

// que eu intercepto lá no script.js para mandar para o
servidor.</script>
<button type="submit" id="btn-salvar">Salvar Produto</button>
</form>
</section>

<script>// Esta seção foi montada para o Desafio JavaScript.
//Aqui está o span do valor total zerado, porque ele serve de
alvo para o script colocar
// a soma totalizadora em tempo real.</script>
<section id="painel-financeiro">
<h3>Resumo Financeiro do Estoque</h3>
<p>Valor Total em Estoque: <strong>R$ <span id="valor-total-
estoque">0.00</span></strong></p>
</section>

<script>// Aqui fica a área de listagem e gerenciamento de dados
do sistema.</script>
<section id="listagem-container">
<h2>Produtos Cadastrados</h2>

<script>// Este é o input de busca dinâmica para o filtro em
tempo real.
//Conforme o usuário digita aqui, o JavaScript esconde as linhas
que não batem com a pesquisa.</script>
<div style="margin-bottom: 15px;">
<label for="busca">🔍 Pesquisar Produto (Nome ou
Categoria):</label>
<input type="text" id="busca" placeholder="Digite para
filtrar..." style="width: 100%; max-width: 400px;">
</div>

<script>// No 'thead' estão os títulos fixos e o 'tbody'
//eu deixei vazio mesmo porque as linhas com as informações do
banco serão geradas pelo JavaScript.</script>
<table id="tabela-produtos">
<thead>
<tr>
<th>ID</th>
<th>Nome</th>
<th>Categoria</th>
<th>Preço</th>
<th>Qtd. Estoque</th>
<th>Fornecedor</th>
<th>Ações</th>
</tr>
</thead>
<tbody id="corpo-tabela">
</tbody>
</table>
</section>
</main>

<footer>

```

```
<p>&copy; 2026 - Projeto Final Ferramentas Web I</p>  
</footer>
```

```
<script>// Por fim, aqui foi chamado o arquivo 'script.js' bem no  
final do body  
//para garantir que o navegador monte toda a tela antes de rodar  
qualquer lógica lógica de programação.</script>  
<script src="script.js"></script>
```

```
</body>  
</html>
```

style.css (O Design e Identidade Visual)

O arquivo style.css é a camada de estilização do projeto, utilizando CSS3 para transformar a estrutura purista do HTML em uma interface profissional, limpa e altamente intuitiva (focada em princípios de UI/UX).

- **O que foi feito nele:**
 - **Paleta de Cores Muted/Profissional:** Aplicação de tons modernos e sóbrios, evitando contrastes agressivos e garantindo uma leitura confortável dos dados.
 - **Alinhamento e Espaçamento:** Uso de regras de posicionamento para garantir que o formulário, os painéis de indicadores e a tabela fiquem organizados harmoniosamente na tela.
 - **Estilização de Tabelas e Botões:** Criação de efeitos visuais sutis (como alteração de cor ao passar o mouse) nos botões de ação e linhas zebradas na tabela para facilitar a leitura das linhas de produtos.

```
* {  
    /* Bom professor, aqui no arquivo style css deixei o seletor  
    universal para fazer um reset global.  
    Zerei as margens e paddings padrões do navegador e apliquei o  
    box-sizing para o  
    tamanho dos blocos não quebrar com espaçamentos. */  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
}  
  
body {  
    /* Aqui eu defini um fundo cinza bem claro e sutil (#f4f6f9)  
    para dar contraste com as seções brancas,  
    usando uma cor escura suave para o texto para não cansar as  
    vistas do usuário. */  
    background-color: #f4f6f9;  
    color: #333;  
    padding: 20px;  
}  
  
header {  
    /* No cabeçalho, adotei uma paleta de cores sóbria e corporativa  
    com o tom azul-escuro (#2c3e50).  
    Coloquei cantos arredondados e uma sombra leve para dar  
    efeito de elevação tridimensional (card). */  
    background-color: #2c3e50;  
    color: white;  
    text-align: center;  
    padding: 20px;  
    border-radius: 8px;  
    margin-bottom: 20px;  
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
```



```

}

main {
    /* No contêiner principal, usei a propriedade CSS Grid para
    organizar o layout das seções
        verticalmente com espaçamentos idênticos e automáticos
    através da propriedade 'gap'. */
    max-width: 1200px;
    margin: 0 auto;
    display: grid;
    gap: 20px;
}

/* Seções (Cards de fundo branco) */
section {
    /* Aqui eu criei o visual estilo 'Card Moderno'. Todas as seções
    ganham fundo totalmente branco,
        cantos arredondados suaves e uma borda sombreada muito leve
    para dar elegância à interface. */
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}

h2, h3 {
    /* Para destacar os títulos de cada bloco, utilizei o mesmo tom
    de azul do cabeçalho
        e inseri uma linha inferior cinza claro para criar uma
    separação visual limpa. */
    color: #2c3e50;
    margin-bottom: 15px;
    border-bottom: 2px solid #ecf0f1;
    padding-bottom: 5px;
}

/* Formulário em Grid */
#form-produto {
    /* Professor, aqui usei CSS Grid com 'auto-fit' e 'minmax'. Isso
    faz com que o formulário seja
        inteligente: os campos ficam lado a lado em telas grandes e se
    organizam sozinhos
        um abaixo do outro em telas menores. */
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
    gap: 15px;
    align-items: flex-end;
}

#form-produto div {
    /* Deixei os grupos de campos como flexbox em coluna para que a
    etiqueta
        (label) fique perfeitamente alinhada acima do seu input. */
    display: flex;

```

```
    flex-direction: column;
}

label {
    /* Estilizei as etiquetas com um tom de cinza escuro, diminuindo
o tamanho e
    aplicando negrito para melhorar a hierarquia visual. */
    font-size: 14px;
    font-weight: 600;
    margin-bottom: 5px;
    color: #555;
}

input {
    /* Nos campos de entrada, apliquei preenchimento interno e
bordas levemente
    suavizadas para dar um aspecto moderno e limpo. */
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 4px;
    font-size: 14px;
    transition: border-color 0.3s;
}

input:focus {
    /* Quando o usuário clica para digitar, a borda do input muda
suavemente para azul.
    Isso melhora a usabilidade (feedback visual). */
    border-color: #3498db;
    outline: none;
}

button {
    /* Para o botão de salvar, usei a cor verde (#27ae60) que remete
a ações
    positivas e de sucesso, deixando a fonte em negrito. */
    background-color: #27ae60;
    color: white;
    border: none;
    padding: 11px 20px;
    font-size: 14px;
    font-weight: bold;
    border-radius: 4px;
    cursor: pointer;
    transition: background 0.3s;
}

button:hover {
    /* Apliquei um efeito hover que escurece o tom de verde quando o
mouse passa por cima,
    indicando que o botão está ativo e clicável. */
    background-color: #219653;
}
```

```

/* Paine! de Destaque Financeiro */
#painel-financeiro {
    /* Para destacar o painel do Desafio JavaScript, mudei o fundo
    para um tom esverdeado bem suave
    e criei uma borda lateral verde grossa para dar a esttica de um
    painel de indicadores (dashboard). */
    background-color: #e8f8f5;
    border-left: 5px solid #2cc990;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

#painel-financeiro p {
    font-size: 18px;
}

#valor-total-estoque {
    /* Destaquei o valor financeiro do estoque aumentando o tamanho
    da fonte para 22px em negrito e na cor verde comercial. */
    color: #27ae60;
    font-size: 22px;
    font-weight: bold;
}

/* Tabela */
#listagem-container {
    /* Professor, coloquei 'overflow-x: auto' para que, se a tabela
    ficar larga
    demais em telas pequenas, apareça uma barra de rolagem local sem
    quebrar o layout do site. */
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
    background: white;
}

th, td {
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}

th {
    /* No cabealho da tabela, usei um fundo cinza bem leve para
    separar o ttulo
    dos dados e apliquei um peso de fonte semi-bold. */
    background-color: #f8f9fa;
    color: #2c3e50;
    font-weight: 600;
}

```

```
}

tr:hover {
    /* Adicionei este efeito de linha: quando o usuário passa o
    mouse sobre
    qualquer linha da tabela, o fundo ganha um tom cinza discreto,
    facilitando o acompanhamento visual dos dados da mercadoria. */
    background-color: #f1f2f6;
}

footer {
    /* Rodapé centralizado com cor cinza atenuada para ficar
    discreto e indicar
    de forma elegante o encerramento da página. */
    text-align: center;
    margin-top: 30px;
    color: #7f8c8d;
    font-size: 14px;
}
```

📁 script.js (A Inteligência do Navegador / Frontend)

O script.js atua como o "cérebro" da interface do usuário no navegador. Ele manipula os elementos do HTML (DOM) e faz a ponte de comunicação instantânea entre o cliente e o servidor Node.js.

O que foi feito nele:

- **Consumo de API (fetch / async-await):** Em vez de salvar dados limitados no navegador via localStorage, o script utiliza funções assíncronas para enviar requisições HTTP (POST) para salvar novos produtos e requisições (GET) para buscar os produtos existentes direto do banco de dados.
- **Renderização Dinâmica:** Função responsável por percorrer a lista de produtos retornada pelo banco, calcular o valor total de cada item (Preço × Quantidade) e desenhar as linhas da tabela dinamicamente.
- **Lógica do Desafio de Estoque:** Uma função matemática acumuladora que soma o valor financeiro de todas as mercadorias inseridas e atualiza o painel principal do site em tempo real.
- **Filtro de Busca em Tempo Real:** Captura o evento de digitação do usuário no campo de busca e oculta as linhas da tabela que não coincidem com o nome ou a categoria digitada, sem precisar recarregar a página.

```
document.addEventListener("DOMContentLoaded", () => {  
  // Professor, usei o evento DOMContentLoaded para garantir que o  
  // script só comece a rodar  
  // depois que o navegador tiver carregado toda a estrutura de tags  
  // da página HTML.  
  const form = document.getElementById("form-produto");  
  const corpoTabela = document.getElementById("corpo-tabela");  
  const valorTotalEstoqueTxt = document.getElementById("valor-total-  
    estoque");  
  const btnSalvar = document.getElementById("btn-salvar");  
  const campoBusca = document.getElementById("busca");  
  
  // Aqui eu defini a URL base do nosso servidor Node.js (Backend) que  
  // criamos na porta 3000.  
  // Todas as requisições do site vão apontar para esse endereço  
  // local.  
  const API_URL = "http://localhost:3000/produtos";  
  
  // FUNÇÃO REQUISITAR (GET): Busca os dados persistidos lá dentro da  
  // tabela do MySQL.  
  async function carregarProdutos() {  
    try {  
      // Professor, utilizei o método 'fetch' com 'async/await' para fazer  
      // uma requisição assíncrona.  
      // O site pede os dados ao servidor e aguarda a resposta sem travar  
      // a tela do usuário.  
      const resposta = await fetch(API_URL);
```

```

const produtos = await resposta.json(); // aqui eu converto a
resposta do servidor para JSON.
renderizarTabela(produtos); // Passo a lista de produtos recebida
para ser desenhada na tela.
} catch (erro) {
console.error("Erro ao buscar produtos do MySQL:", erro);
}
}

// FUNÇÃO RENDERIZAR: Responsável por desenhar as linhas da tabela e
calcular o desafio.
function renderizarTabela(produtos) {
corpoTabela.innerHTML = ""; // Limpo a tabela para não duplicar os
itens na tela.
let valorTotalGeral = 0; // Criei esta variável acumuladora para o
Desafio JavaScript.

produtos.forEach((produto, index) => {
// Como mapeamos direto do banco, puxo o campo 'quantidade_estoque'
exatamente igual à coluna do MySQL.
const quantidade = produto.quantidade_estoque || produto.quantidade
|| 0;

// Aqui eu faço o cálculo matemático individual: Preço do produto
multiplicado pela Quantidade.
const valorTotalItem = produto.preco * quantidade;

// Somo o valor desse item ao montante geral do estoque (Lógica do
Desafio).
valorTotalGeral += valorTotalItem;

// Crio a linha estruturada no HTML e injeto os dados do objeto
vindos do banco de dados.
const tr = document.createElement("tr");
tr.innerHTML = `
<td>${produto.id || index + 1}</td>
<td>${produto.nome}</td>
<td>${produto.categoria}</td>
<td>R$ ${parseFloat(produto.preco).toFixed(2)}</td>
<td>${quantidade}</td>
<td>${produto.fornecedor}</td>
<td>
<button class="btn-excluir" data-id="${produto.id}"
style="background-color: #e74c3c; padding: 5px 10px; color: white;
border: none; cursor: pointer;">Excluir</button>
</td>
`;
corpoTabela.appendChild(tr);
});

// Professor, aqui eu atualizo o painel do Desafio na tela, travando
em duas casas decimais com o 'toFixed(2)'.
valorTotalEstoqueTxt.textContent = valorTotalGeral.toFixed(2);
}

```

```

// EVENTO DE SUBMIT (POST): Intercepta o clique no botão Salvar e
// manda os dados para o MySQL.
form.onsubmit = async (e) => {
  e.preventDefault(); // Comando crucial para impedir que a página
  // recarregue e perca o fluxo.

  // Capturo os valores digitados pelo usuário em cada campo do
  // formulário.
  const nome = document.getElementById("nome").value;
  const categoria = document.getElementById("categoria").value;
  const preco = parseFloat(document.getElementById("preco").value);
  const quantidade_estoque =
    parseInt(document.getElementById("quantidade").value);
  const fornecedor = document.getElementById("fornecedor").value;

  // Monto um objeto estruturado em JavaScript com os dados coletados.
  const novoProduto = { nome, categoria, preco, quantidade_estoque,
    fornecedor };

  try {
    // Faço uma requisição POST enviando o produto no corpo da mensagem
    // (body) em formato de texto JSON.
    const resposta = await fetch(API_URL, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(novoProduto)
    });

    if (resposta.ok) {
      form.reset(); // Se o servidor gravou com sucesso no MySQL, eu limpo
      // os campos do formulário.
      carregarProdutos(); // E chamo a função para recarregar a tabela
      // atualizada com o novo item do banco.
    }
  } catch (erro) {
    console.error("Erro ao salvar produto no banco:", erro);
  }
};

// LÓGICA DA BUSCA DINÂMICA: Filtro local na tabela em tempo real.
if (campoBusca) {
  campoBusca.addEventListener("input", (e) => {
    // Converto o termo digitado para letras minúsculas para a busca não
    // diferenciar maiúsculas de minúsculas.
    const termoBusca = e.target.value.toLowerCase();
    const linhas = corpoTabela.querySelectorAll("tr");

    linhas.forEach(linha => {
      // Capturo o texto contido nas células de Nome (Coluna 1) e
      // Categoria (Coluna 2).
      const nomeProduto = linha.cells[1].textContent.toLowerCase();

```

```
const categoriaProduto = linha.cells[2].textContent.toLowerCase();

// Se o termo digitado existir no nome ou na categoria, a linha
continua visível.
// Caso contrário, eu mudo o display para 'none' para ocultar o
registro temporariamente.
if (nomeProduto.includes(termoBusca) ||
categoriaProduto.includes(termoBusca)) {
linha.style.display = "";
} else {
linha.style.display = "none";
}
});
});
}

// INICIALIZAÇÃO: Assim que o portal abre, ele dispara o gatilho
para trazer os dados guardados no MySQL.
carregarProdutos();
});
```


server.js (O Motor do Servidor / Backend)

O server.js é o coração do Backend do projeto, rodando fora do navegador através do ambiente de execução **Node.js**. Ele atua como um intermediário de segurança e persistência, recebendo as ordens da página web e executando os comandos na tela preta do MySQL. Ele cumpre a mesma função dos scripts em Java Web ensinados em aula, utilizando o ecossistema JavaScript de alta performance.

O que foi feito nele:

- **Criação de Servidor HTTP (Express):** Configuração de uma API local rodando na porta 3000 capaz de escutar requisições de rede.
- **Habilitação de Segurança (CORS):** Configuração que permite que o navegador (index.html) faça requisições ao servidor de backend sem bloqueios de segurança do navegador.
- **Driver de Conexão MySQL (mysql2):** Equivalente ao driver *Connector/J* usado no Java, este módulo abre uma ponte segura de autenticação com o banco de dados local (localhost) usando o usuário root e a senha definida.
- **Rota de Cadastro (POST /produtos):** Recebe o objeto do produto enviado pelo script.js, valida os dados e executa o comando SQL compilado `INSERT INTO produtos (...) VALUES (?, ?, ?, ?, ?)` para gravar de forma permanente na tabela.
- **Rota de Consulta (GET /produtos):** Executa a instrução nativa `SELECT * FROM produtos` no MySQL, captura as linhas retornadas pela tela preta do banco, converte-as para o formato padrão da web (JSON) e responde ao navegador para exibição na tabela.

```
• const express = require("express");
• const mysql = require("mysql2");
• const cors = require("cors");
•
• // Professor, inicializei o Express para gerenciar as rotas
  HTTP e ativei o CORS
• // para que o navegador (Frontend) possa fazer requisições de
  rede para o Servidor sem bloqueios de segurança.
• const app = express();
• app.use(cors());
• app.use(express.json()); // Configuro o Express para entender
  dados enviados no formato JSON.
•
• // CONFIGURAÇÃO DO BANCO DE DADOS: Equivalente ao Driver JDBC
  do Java.
• // Estabeleço a conexão com o banco local apontando para as
  minhas credenciais do MySQL.
• const db = mysql.createConnection({
•   host: "localhost",
•   user: "root",
•   password: "", // Coloque aqui a senha do seu MySQL caso possua
  uma
```

- database: "controle_estoque" // Nome do banco de dados que criamos na tela preta
- });
-
- // TESTE DE CONEXÃO: Executado assim que o comando 'node server.js' roda no CMD.
- db.connect((erro) => {
- if (erro) {
- console.error("Erro crítico ao conectar no MySQL:", erro);
- return;
- }
- console.log(" 🍷 Conectado com sucesso na tela preta do MySQL!");
- });
-
- // ROTA GET /produtos: Busca as informações salvas para preencher a tabela do site.
- app.get("/produtos", (req, res) => {
- // Professor, aqui eu executo uma instrução SQL nativa para ler todas as linhas da tabela.
- const querySQL = "SELECT * FROM produtos";
-
- db.query(querySQL, (erro, resultados) => {
- if (erro) {
- console.error("Erro ao buscar dados do banco:", erro);
- return res.status(500).json({ mensagem: "Erro no servidor" });
- }
- // Retorno os dados capturados do banco de dados convertidos em formato JSON para o Frontend.
- res.json(resultados);
- });
- });
-
- // ROTA POST /produtos: Recebe os dados do formulário e insere um novo produto no MySQL.
- app.post("/produtos", (req, res) => {
- // Desestruturo o objeto enviado pelo script.js para pegar cada dado individual do produto.
- const { nome, categoria, preco, quantidade_estoque, fornecedor } = req.body;
-
- // Professor, usei interrogações (?) na query por questões de segurança.
- // Isso evita ataques de SQL Injection, que é uma das maiores vulnerabilidades de Backend do mercado.
- const querySQL = "INSERT INTO produtos (nome, categoria, preco, quantidade_estoque, fornecedor) VALUES (?, ?, ?, ?, ?)";
- const valores = [nome, categoria, preco, quantidade_estoque, fornecedor];
-
- db.query(querySQL, valores, (erro, resultado) => {
- if (erro) {
- console.error("Erro ao inserir produto no MySQL:", erro);

- return res.status(500).json({ mensagem: "Erro ao salvar registro" });
- }
- // Se a gravação deu certo, respondo ao navegador com o status 201 (Criado com Sucesso).
- res.status(201).json({ mensagem: "Produto cadastrado com sucesso!", id: resultado.insertId });
- });
- });
-
- // INICIALIZAÇÃO DO SERVIDOR: Configuração da porta de escuta da API.
- const PORTA = 3000;
- app.listen(PORTA, () => {
- console.log(`🚀 Servidor rodando na porta \${PORTA}. Pronto para receber dados!`);
- });

Professor, para testar o trabalho no cmd coloque :

cd (espaço)

"C:\Users\diego\OneDrive\Documentos\NetBeansProjects\ControleEstoque\pasta onde está salvo o seu arquivo no net bens"

(BANCO DE DADOS MYSQL)

"Professor, para garantir que os produtos do estoque permaneçam salvos mesmo após o sistema ou o computador serem desligados, integramos a aplicação ao banco de dados relacional MySQL. Criei um banco de dados chamado controle_estoque e, dentro dele, estruturamos uma tabela com chaves primárias e tipos de dados específicos para cada atributo do produto."

Script de Criação (DDL - Data Definition Language)

Este é o código SQL que utilizei na linha de comando (ou no MySQL Workbench/phpMyAdmin) para construir a nossa base de dados. O senhor pode copiar este código e colar no mysql do senhor.

```
CREATE DATABASE controle_estoque;

USE controle_estoque;

CREATE TABLE IF NOT EXISTS produtos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(150) NOT NULL,
  categoria VARCHAR(100) NOT NULL,
  preco DECIMAL(10, 2) NOT NULL,
  quantidade_estoque INT NOT NULL DEFAULT 0,
  fornecedor VARCHAR(150) NOT NULL,
  data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Aqui está a explicação detalhada de cada campo que criei:

- **id (INT AUTO_INCREMENT PRIMARY KEY):**

Defini o ID como chave primária com auto-incremento. Isso garante que cada produto inserido ganhe um número identificador único e exclusivo gerado automaticamente pelo próprio MySQL, o que evita qualquer duplicidade de registros.

- **nome (VARCHAR(150)) e categoria (VARCHAR(100)):**

Usei o tipo VARCHAR por se tratar de campos de texto de tamanho variável. O limite de caracteres foi planejado para economizar memória no servidor, sendo mais do que suficiente para armazenar nomes de produtos complexos e suas respectivas classificações.

- **preco (DECIMAL(10, 2)):**

Professor, essa escolha é muito importante. Para valores monetários (dinheiro), não podemos usar o tipo 'Float' ou 'Double' devido a arredondamentos imprecisos. O tipo DECIMAL(10,2) garante precisão absoluta, permitindo números de até 10 dígitos no total, sendo exatamente 2 deles reservados para os centavos.

- **quantidade_estoque (INT):**

Como a contagem de mercadorias em estoque lida apenas com unidades inteiras (não existem 1.5 teclados no estoque), o tipo INT (Inteiro) é o mais adequado e performático para o processamento matemático.

- **fornecedor (VARCHAR(150)):**

Campo de texto projetado para registrar o nome da empresa ou distribuidora responsável pelo fornecimento do insumo.

- **data_cadastro (TIMESTAMP):**

Incluí este campo como uma boa prática de auditoria. O modificador DEFAULT CURRENT_TIMESTAMP faz com que o próprio MySQL carimbe o dia e o horário exato em que o produto foi cadastrado, sem que o nosso código JavaScript precise enviar essa informação.